

NTU-DAVID-RAG

Setup & Deployment Manual

Single-GPU and Two-GPU Docker installations

Abstract

This manual is a step-by-step guide to installing and running the NTU-DAVID-RAG service from a clean machine. It covers prerequisites and GPU/Docker setup for Linux, Windows, and macOS; cloning the repository; configuring secrets; and two deployment options — a **two-GPU** default (the 7B embedder on one card, the answer LLM on another) and a **single-GPU** mode that fits the whole stack on one card. Every step lists the exact commands.

Contents

1	What you are installing	1
2	System requirements	1
2.1	Hardware	1
2.2	Software	2
2.3	GPU + Docker setup by operating system	2
3	Get the code	3
4	Configure secrets and ports	3
5	Option A — Two-GPU deployment (default)	4
6	Option B — Single-GPU deployment	4
7	First run and verification	5
8	Day-to-day operations	5
9	Troubleshooting	6
10	Quick reference	6

1 What you are installing

NTU-DAVID-RAG is a containerized retrieval-augmented-generation service for economics papers and their Fortran model code. The stack is orchestrated with Docker Compose and exposes a single web dashboard behind an Nginx proxy. It has four moving parts:

- **embedding-server** — a GPU container that loads the text embedder and the cross-encoder reranker once and serves them over HTTP.
- **rag-web** — the Flask/Gunicorn application (ingestion, retrieval, generation, auth). It is CPU-only except for MinerU (PDF layout/OCR), which uses the GPU transiently during ingestion.

- **answer LLM** — an Ollama service (e.g. `gemma4:31b` or `gemma3:12b`) that writes the grounded answers.
- **auth-db + proxy** — PostgreSQL for users/sessions and an Nginx reverse proxy that publishes the dashboard on one port.

Two ways to run it:

- **Two-GPU (default).** The 7B embedder owns one GPU; the `gemma4:31b` LLM runs as a separate Ollama stack on another GPU. Highest quality. See Section 5.
- **Single-GPU.** An override collapses the embedder, reranker, MinerU, and an in-stack LLM onto one card, swapping in lighter models. See Section 6.

2 System requirements

2.1 Hardware

A CUDA-capable NVIDIA GPU is required — model inference does not run on CPU in any practical configuration. Table 1 lists the rough VRAM and disk budgets.

Mode	GPU / VRAM	Disk
Two-GPU	Card A $\geq$ 24 GB (embedder <code>gte-Qwen2-7B fp16</code> $\sim$ 15 GB + reranker $\sim$ 2 GB + transient MinerU); Card B $\geq$ 24 GB (LLM <code>gemma4:31b Q4</code> $\sim$ 18 GB)	$\sim$ 60 GB (weights + images)
Single-GPU	One card $\geq$ 24 GB with a 12B LLM (<code>gemma3:12b</code> $\sim$ 9 GB + small embedder + reranker + MinerU); $\geq$ 40 GB if you run <code>gemma4:31b</code> on the same card	$\sim$ 40 GB (weights + images)

Table 1: Approximate hardware budgets. RAM $\geq$ 16 GB and a fast SSD are recommended; the first start downloads several GB of model weights.

2.2 Software

- **Git.**
- **Docker Engine** ($\geq$ 24) with the **Compose v2** plugin (the `docker compose` subcommand), or Docker Desktop.
- **NVIDIA GPU driver** new enough for CUDA 12.x.
- **NVIDIA Container Toolkit** — lets containers see the GPU.

2.3 GPU + Docker setup by operating system

Linux (Ubuntu/Debian) — recommended. Install Docker Engine, then the NVIDIA Container Toolkit:

```
# 1) Docker Engine + Compose plugin
curl -fsSL https://get.docker.com | sh
sudo usermod -aG docker "$USER"    # log out/in afterwards
```

```
# 2) NVIDIA Container Toolkit (official repo)
curl -fsSL https://nvidia.github.io/libnvidia-container/gpgkey \
  | sudo gpg --dearmor -o /usr/share/keyrings/nvidia-container-toolkit-keyring.gpg
curl -s -L https://nvidia.github.io/libnvidia-container/stable/deb/nvidia-container-toolkit.
  ↪ list \
  | sed 's#deb https://#deb [signed-by=/usr/share/keyrings/nvidia-container-toolkit-keyring.
  ↪ gpg] https://#g' \
  | sudo tee /etc/apt/sources.list.d/nvidia-container-toolkit.list
sudo apt-get update && sudo apt-get install -y nvidia-container-toolkit

# 3) Wire the toolkit into Docker and restart
sudo nvidia-ctk runtime configure --runtime=docker
sudo systemctl restart docker
```

Verify the GPU is visible inside a container:

```
docker run --rm --gpus all nvidia/cuda:12.4.1-base-ubuntu22.04 nvidia-smi
```

You should see your GPU(s). (The NVIDIA *driver* itself is installed on the host the usual way for your distro, e.g. `sudo apt-get install nvidia-driver-550`.)

Windows 10/11 — via WSL2.

1. Install WSL2: `wsl --install` in an admin PowerShell, then reboot.
2. Install the **NVIDIA Windows driver** that includes WSL2/CUDA support (the standard Game-Ready/Studio driver does). Do *not* install a separate driver inside WSL.
3. Install **Docker Desktop** and enable *Settings* → *Resources* → *WSL integration*.
4. In a WSL2 (Ubuntu) shell, verify: `docker run --rm --gpus all nvidia/cuda:12.4.1-base-ubuntu22.04 nvidia-smi`.

Clone and run the project *inside the WSL2 filesystem* (e.g. under `~/` in Ubuntu, not `/mnt/c/...`) for correct file permissions and speed.

macOS — not supported for the GPU stack. Docker Desktop on macOS cannot pass through an NVIDIA GPU (there is no CUDA on Mac). The embedder, reranker, LLM, and MinerU therefore cannot run locally. Use a Linux host or a remote GPU server for the service; a Mac can act as a *client* (open the dashboard in a browser, optionally over an SSH tunnel: `ssh -L 3006:localhost:3006 user@gpu-host`).

3 Get the code

```
git clone https://github.com/desmond65/NTU-DAVID-RAG.git
cd NTU-DAVID-RAG
```

All deployment files live under `docker/`.

4 Configure secrets and ports

The stack reads its configuration from `docker/.env`. Create it from the template and set the two required secrets:

```
cd docker
cp .env.example .env
```

```
# Generate strong secrets and paste them into .env:
openssl rand -hex 24    # -> AUTH_DB_PASSWORD
openssl rand -hex 32    # -> SESSION_SECRET
```

Key settings in `docker/.env` (defaults shown):

Variable	Default	Meaning
UNIFIED_PORT	3006	External dashboard port.
EMBEDDING_GPU	1	Physical GPU index for the embedder/reranker.
RAG_WEB_GPU	0	GPU index for MinerU during ingestion.
MINERU_HYBRID_BATCH_RATIO	8	Caps MinerU's batch size to avoid OOM on a shared card.
EMBEDDING_MODEL	gte-Qwen2-7B	Embedder (must match what populated the DB).
AUTH_DB_PASSWORD	—	Required. PostgreSQL password.
SESSION_SECRET	—	Required. Session/CSRF signing key.
ALLOW_REGISTRATION	true	Set false after creating accounts.
SESSION_COOKIE_SECURE	false	Set true when behind HTTPS.

Table 2: Core environment variables (full list in `docker/.env.example`).

5 Option A — Two-GPU deployment (default)

The embedder/reranker run on `EMBEDDING_GPU`; the answer LLM (`gemma4:31b`) runs as a separate Ollama service on its own GPU.

Step 1 — Start the answer LLM. The standalone Ollama compose pins the model to GPU index 2; edit `NVIDIA_VISIBLE_DEVICES/CUDA_VISIBLE_DEVICES` in the file to pick another card.

```
cd docker/local_llm
docker compose -f docker-compose.gemma4_31b_ollama.yml up -d
# First start pulls gemma4:31b (~19 GB) and warms it. API on :11434.
docker logs -f gemma4_ollama      # wait for "Model ready."
```

Step 2 — Start the RAG stack. From `docker/`, with your `.env` in place:

```
cd ..          # back to docker/
./up.sh --build # builds images, starts auth-db, embedding-server,
               # rag-web, and the proxy (foreground; logs to log.txt)
# Detached instead: ./up.sh -d --build
```

The first boot downloads the 7B embedder (~29 GB) and reranker into `docker/./hf_cache`; the `embedding-server` healthcheck has a ~10 minute grace window while this happens. `rag-web` starts only once the embedder is healthy.

Step 3 — Confirm the LLM connection. `rag-web` reaches the LLM via `docker/RAG/.env` (`LOCAL_LLM_BASE_URL`, default `http://host.docker.internal:11434/v1`, and `LLM_MODEL_NAME=gemma4:31b`). Adjust if your LLM runs elsewhere.

Continue at Section 7.

6 Option B — Single-GPU deployment

A compose override (`docker/docker-compose.one-gpu.yml`) runs *everything* on one card: a lighter embedder, the reranker, MinerU, and an in-stack Ollama LLM. Your default two-GPU setup is untouched — this is an opt-in launcher. Full notes: `docker/README.one-gpu.md`.

Step 1 — (Optional) pick models and token. Defaults: embedder `embeddinggemma-300m`, LLM `gemma3:12b-it-qat`. `embeddinggemma` is *gated* on Hugging Face — either accept its license and set a token, or switch to a non-gated embedder. Put any overrides in `docker/.env`:

```
# docker/.env (single-GPU knobs; all optional)
HF_TOKEN=hf_xxx # needed only for embeddinggemma
ONE_GPU_EMBEDDING_MODEL=BAAI/bge-large-en-v1.5 # non-gated alternative
ONE_GPU_LLM_MODEL=gemma3:12b-it-qat
# OLLAMA_CONTEXT_LENGTH=32768 # cap the LLM context to fit VRAM
```

Step 2 — Launch on a chosen card.

```
cd docker
GPU=0 ./up.one-gpu.sh --build # GPU=<physical index> selects the card
# Detached: GPU=0 ./up.one-gpu.sh -d --build
```

First boot builds the embedding image, pulls the LLM (~8–19 GB), and loads the models; the in-stack `ollama` healthcheck waits for the pull to finish before `rag-web` starts.

Note — switching embedders is non-destructive. Each embedder writes to its own Qdrant collections (`rag-embeddings_<model>`), so different-dimension models coexist; you do not need to wipe the database when changing `ONE_GPU_EMBEDDING_MODEL`. The new model’s collections start empty, so re-ingest your papers under it.

7 First run and verification

1. **Watch startup.** `docker compose ps` shows service health; `docker logs -f embedding-server` shows the embedder loading. Expect several minutes on a cold cache.
2. **Open the dashboard:** `http://<host>:3006/`. Over SSH: `ssh -L 3006:localhost:3006 user@host` then browse `http://localhost:3006/`.
3. **Create an account.** Register on the login page (open registration is on by default). Set `ALLOW_REGISTRATION=false` in `.env` and restart once your accounts exist.
4. **Ingest a paper.** Upload a PDF (and, optionally, its Fortran source). Ingestion is synchronous — PDF parsing, enrichment, code digestion, chunking, embedding — and can take several minutes per paper on the first run.
5. **Ask a question.** Each paper’s vectors are isolated per user; answers cite paper titles and are grounded in retrieved context.

8 Day-to-day operations

Run these from `docker/` (add `-f docker-compose.yml -f docker-compose.one-gpu.yml` for single-GPU mode, or just use the matching up script).

Task	Command
Start (two-GPU)	<code>./up.sh -d</code>
Start (single-GPU)	<code>GPU=0 ./up.one-gpu.sh -d</code>
Status / health	<code>docker compose ps</code>
Follow a service log	<code>docker logs -f rag-web</code>
Rebuild after a code change	<code>./up.sh -d --build</code>
Stop (keep data)	<code>docker compose down</code>
Stop single-GPU stack	<code>docker compose -f docker-compose.yml -f docker-compose.one-gpu.yml down</code>
GPU usage	<code>nvidia-smi</code>

Table 3: Common operational commands.

Persistent state lives in the repository tree and survives down: `db/` (Qdrant vectors), `data/` (uploaded papers), `auth_db/` (PostgreSQL), and `hf_cache/` (downloaded model weights).

9 Troubleshooting

- **“could not select device driver ... gpu”** — the NVIDIA Container Toolkit is not configured. Re-run `sudo nvidia-ctk runtime configure --runtime=docker` and restart Docker; confirm with the `--gpus all` test in Section 2.
- **Port 3006 already in use** — set a different `UNIFIED_PORT` in `.env` and restart.
- **embedding-server stuck “starting”** — it is still downloading the 7B weights (~29 GB). Watch `docker logs -f embedding-server`; the healthcheck allows ~10 min.
- **GPU out-of-memory during ingestion** — lower `MINERU_HYBRID_BATCH_RATIO` ($8 \rightarrow 4 \rightarrow 2$), and in single-GPU mode lower `OLLAMA_CONTEXT_LENGTH`.
- **Ingestion times out (504)** — very large Fortran files make a single upload exceed the proxy timeout; the bundled Nginx/Gunicorn timeouts are 3600s, which suits large files but still bounds them.
- **Empty answers / LLM errors** — verify the LLM is reachable: `curl http://localhost:11434/api/tags` should list the model named in `LLM_MODEL_NAME`.
- **Gated-model 401 (single-GPU)** — set `HF_TOKEN` in `.env` for `embeddinggemma`, or use the non-gated `BAAI/bge-large-en-v1.5`.

10 Quick reference

```
# ---- Two-GPU (default) -----
git clone https://github.com/desmond65/NTU-DAVID-RAG.git
cd NTU-DAVID-RAG/docker && cp .env.example .env      # set AUTH_DB_PASSWORD + SESSION_SECRET
( cd local_llm && docker compose -f docker-compose.gemma4_31b_ollama.yml up -d )
./up.sh -d --build
# open http://<host>:3006/

# ---- Single-GPU -----
cd NTU-DAVID-RAG/docker && cp .env.example .env      # set secrets (+ HF_TOKEN or bge-large)
GPU=0 ./up.one-gpu.sh -d --build
# open http://<host>:3006/
```